Create Database and Import Data

Mongosh

use salesData

Create file sales.json:

[

{ "_id": 1, "customerId": "A123", "product": "Laptop", "quantity": 1, "price": 1200, "date": "2023-01-15" },

{ "_id": 2, "customerId": "B456", "product": "Mouse", "quantity": 2, "price": 25, "date": "2023-01-20" },

{ "_id": 3, "customerId": "A123", "product": "Keyboard", "quantity": 1, "price": 75, "date": "2023-02-01" },

{ "_id": 4, "customerId": "C789", "product": "Laptop", "quantity": 1, "price": 1300, "date": "2023-02-10" },

{ "_id": 5, "customerId": "B456", "product": "Monitor", "quantity": 1, "price": 300, "date": "2023-02-15" },

{ "_id": 6, "customerId": "A123", "product": "Webcam", "quantity": 1, "price": 50, "date": "2023-03-01" },

{ "_id": 7, "customerId": "D012", "product": "Headphones", "quantity": 1, "price": 150, "date": "2023-03-10" },

{ "_id": 8, "customerId": "C789", "product": "Keyboard", "quantity": 1, "price": 80, "date": "2023-03-15" },

{ "_id": 9, "customerId": "E345", "product": "Mouse", "quantity": 2, "price": 30, "date": "2023-03-20" },

{ "_id": 10, "customerId": "A123", "product": "Laptop", "quantity": 1, "price": 1250, "date": "2023-04-01" }

]

mongoimport --db salesData --collection transactions --file sales.json --jsonArray

Step 1: Total Spending per Customer

Command: db.transactions.aggregate([

```
 {
   $group: {
     _id: "$customerId",
     totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
   }
 }
])
```

```
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   }
| ])
[
  { _id: 'E345', totalSpending: 60 },
  { _id: 'B456', totalSpending: 350 },
  { _id: 'A123', totalSpending: 2575 },
  { _id: 'C789', totalSpending: 1380 },
  { _id: 'D012', totalSpending: 150 }
]
```

Explanation: The $group stage is used to group documents based on the customerId. For each group, the totalSpending is calculated by multiplying quantity and price for each transaction and then summing the results using $sum. This provides the total amount spent by each customer.

Step 2: Filtering Customers with Spending Greater Than a Threshold

Command: db.transactions.aggregate([

```
 {
   $group: {
     _id: "$customerId",
     totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
   }
 },
 {
   $match: {
```

```
    totalSpending: { $gt: 500 }
  }
 }
])
```

```
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   },
|   {
|     $match: {
|       totalSpending: { $gt: 500 }
|     }
|   }
| ])
|
[
  { _id: 'A123', totalSpending: 2575 },
  { _id: 'C789', totalSpending: 1380 }
]
```

Explanation: The $match stage filters the aggregated results from the previous stage. It only includes customers whose totalSpending is greater than 500. This helps focus on high-value customers.

Step 3: Projecting Relevant Fields and Renaming

Command:

```
db.transactions.aggregate([
 {
  $group: {
   _id: "$customerId",
   totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
  }
 },
 {
  $match: {
   totalSpending: { $gt: 500 }
  }
 },
 {
  $project: {
   _id: 0,
   customerId: "$_id",
   total: "$totalSpending"
  }
```

```
  }
])
```

```
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   },
|   {
|     $match: {
|       totalSpending: { $gt: 500 }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       customerId: "$_id",
|       total: "$totalSpending"
|     }
|   }
| ])
|
[
  { customerId: 'A123', total: 2575 },
  { customerId: 'C789', total: 1380 }
]
```

Explanation: The $project stage reshapes the output documents. The _id field is excluded by setting _id: 0. The customerId field is recreated from the _id, and totalSpending is renamed to total. This makes the output cleaner and easier to read.

Step 4: Unwinding an Array (Hypothetical Scenario)

```
Command: db.transactions.updateMany(
  {},
  [
    {
      $set: {
        categories: {
          $cond: {
            if: { $gt: ["$price", 500] },
            then: ["expensive", "electronics"],
            else: ["cheap", "accessories"]
          }
        }
      }
    }
  ]
)
```

Command 2:

```
db.transactions.aggregate([
  {
```

```
    $unwind: "$categories"
  },
  {
    $group: {
      _id: "$categories",
      count: { $sum: 1 }
    }
  }
])
```

```
salesData> db.transactions.updateMany(
|   {},
|   [
|     {
|       $set: {
|         categories: {
|           $cond: {
|             if: { $gt: ["$price", 500] },
|             then: ["expensive", "electronics"],
|             else: ["cheap", "accessories"]
|           }
|         }
|       }
|     }
|   ]
| )
|
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 10,
  modifiedCount: 10,
  upsertedCount: 0
}
salesData> db.transactions.aggregate([
|   {
|     $unwind: "$categories"
|   },
|   {
|     $group: {
|       _id: "$categories",
|       count: { $sum: 1 }
|     }
|   }
| ])
|
[
  { _id: 'accessories', count: 7 },
  { _id: 'cheap', count: 7 },
  { _id: 'expensive', count: 3 },
  { _id: 'electronics', count: 3 }
]
```

Explanation: The $unwind stage is used to deconstruct the categories array field, creating a separate document for each element in the array. After that, the $group stage counts how many times each category appears. This is useful for analyzing data stored in arrays.

Step 5: Sorting Results

Command:

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$customerId",
```

```
      totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
    }
  },
  {
    $match: {
      totalSpending: { $gt: 500 }
    }
  },
  {
    $project: {
      _id: 0,
      customerId: "$_id",
      total: "$totalSpending"
    }
  },
  {
    $sort: {
      total: -1
    }
  }
])
```

```
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   },
|   {
|     $match: {
|       totalSpending: { $gt: 500 }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       customerId: "$_id",
|       total: "$totalSpending"
|     }
|   },
|   {
|     $sort: {
|       total: -1
|     }
|   }
| ])
|
[
  { customerId: 'A123', total: 2575 },
  { customerId: 'C789', total: 1380 }
]
```

Explanation: The $sort stage sorts the final results in descending order based on the total field. This allows us to easily identify customers with the highest spending.

Challenges & Solutions

1. Incorrect total calculation due to wrong field names or data types.
2. $match stage not filtering properly because of incorrect field reference.
3. $unwind errors when the target field is not an array or does not exist.
4. Confusion when _id still appears in the result.
5. Performance issues when working with larger datasets.

Solutions

1. Carefully verify field names and ensure numeric values are used in calculations.
2. Double-check the conditions and field names used in the $match stage.
3. Ensure the field exists and is an array before using $unwind (use $set if necessary).
4. Use $project with _id: 0 to remove the _id field.
5. Optimize performance by placing $match early in the pipeline and adding indexes to frequently queried fields.